Smart MCQ Solver Challenge - Project Report

Course: BSDA2001P - Introduction to Deep Learning and Generative AI

Name: Ashvin Ganeshrao Ambatwar

Email Id: 24f2009283@ds.study.iitm.ac.in

Kaggle competition: smart-mcq-solver-challenge

Repository: github.com/ashwinambatwariitm/dlgenai-t22026 (private)

Deployed model: huggingface.co/ashwinambatwariitm/dlgenai-t22026-mcq

Live demo: https://smart-mcq-solver-dlgenai.streamlit.app/

---

1. Abstract / Executive Summary

Problem. Answer five-option multiple-choice questions. For each of 500 test questions the system submits three ranked letters, scored by MAP@3.

Methods. Nine systems were compared under one protocol:

1. Three baselines TF-IDF with Logistic Regression and with a linear SVM, a zero-shot MiniLM ranker
2. Three from-scratch encoders sharing one scoring head (BiLSTM, text-CNN, mini transformer),
3. A fully fine-tuned `bert-base-uncased`
4. A retrieval-augmented pipeline using Qwen2.5-0.5B-Instruct
5. Weighted Borda ensemble.

Results. The ensemble led on held-out validation at **MAP@3 0.8858**, ahead of the fine-tuned transformer (0.8729), the BiLSTM (0.8633), the mini transformer (0.8471), the text-CNN (0.8462) and RAG (0.8208).

Best Kaggle score: **0.75664**, against a 0.73 qualifying cutoff.

Main learning. The dataset contains two properties that make the raw scores look better than the actual question understanding. First, the answer-text artifact baseline achieved 0.7954 MAP@3 without reading the question. This means that the text of the answer choices itself contains useful information about which option is correct. Therefore, comparing BERT only with the constant baseline of 0.3937 would give an overly optimistic view of the improvement. BERT achieved 0.8729, but its improvement over the question-blind baseline is about 0.078.

Second, the validation set and Kaggle test set have different compositions. The test set contains a large number of questions that are repeated from the training data, while the

leakage-free validation split was designed to remove near-duplicate questions. Therefore, the validation score mainly measures generalization, whereas the leaderboard score is also influenced by the ability to recognize previously seen questions. This is why validation and leaderboard performance do not always move in the same direction.

---

2. Introduction

2.1 Problem Statement

Given a `prompt` and five candidate answers A to E, identify the correct option. Training has 2,000 labelled questions, test has 500 unlabelled. Submissions rank three letters and are scored by MAP@3: 1.0 if the correct letter is first, 0.5 if second, 1/3 if third, 0 otherwise.

2.2 Project Objective

1. Exceed the 0.73 qualifying cutoff.
2. Compare three model families - a fine-tuned pretrained transformer, a custom architecture trained from random initialisation, and a sequence model - under one metric and one split. The from-scratch family was later extended to three encoders (recurrent, convolutional, self-attention) in an identical harness, so architecture could be separated from pretraining.
3. Gain practical command of PyTorch, Hugging Face Transformers, Weights and Biases, and deployment.
4. Establish an evaluation protocol trustworthy enough to support model selection, and diagnose it when it proves untrustworthy.

2.3 Report Structure

Section 3 covers the dataset and the two flaws that shape everything else. Section 4 covers tokenisation, Section 5 the models, Section 6 performance and the validation-leaderboard divergence, Section 7 conclusions.

---

3. Dataset and Preprocessing

3.1 Dataset Description

2,000 training and 500 test questions with columns `id`, prompt, A-E, and answer on train only. No missing values, exactly 5 options throughout. Topics span philosophy, physics and general academic knowledge.

3.2 Exploratory Data Analysis

Answers are mildly imbalanced, so a constant guess carries signal and must be measured rather than assumed to be 20%:

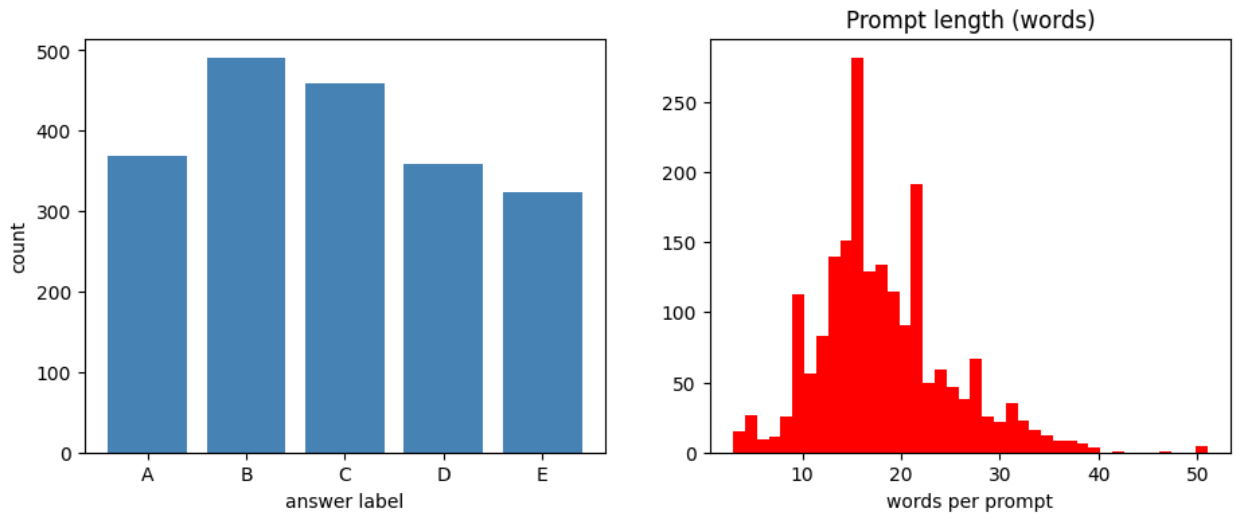


Figure: `Dataset answer distribution figures`

answer	A	B	C	D	E
count	369	490	459	358	324
share	18.4%	24.5%	23.0%	17.9%	16.2%

The most frequent triple `B C D` scores 0.4088, above the 0.3937 of a naive A B C. Prompts are short (mean 18.1 words, max 51), options are longer (longest per question: mean 30.5, max 118), which drove the sequence-length decisions in Section 4.

Finding 1 - the dataset is mostly duplicates. Clustering prompts by sentence-embedding cosine similarity at 0.95 collapses 2,000 rows into **613 distinct questions**. 306 appear exactly once; 307 appear 2 to 21 times. One question appears 21 times in 17 phrasings, every copy answered B. This shows that the effective diversity of the dataset is much smaller than the total number of rows.

Finding 2 - correct answers are identifiable by their text. Across all 2,000 questions there are 313 distinct correct-answer texts and 1,271 distractor texts, and the **intersection is empty**. Not one correct answer is ever reused as a wrong option. Section 6.3 measures what that is worth: 0.7954 MAP@3 without reading the question.

3.3 Data Preprocessing

Cleaning was minimal. The substantive decision was the **validation split**. Given Finding 1, a random holdout leaves most validation questions with a twin in training. The notebook measures this rather than asserting it:

split	validation questions with a near-duplicate in train
random 80/20	79.5%
cluster-based (used)	**0.0%**

The fix embeds all 2,000 prompts with `all-MiniLM-L6-v2`, unions any pair at cosine >= 0.95 into a cluster (613 clusters), then assigns **whole clusters** to one side until about 20% of rows is reached. This gives 1,600 / 400 rows from 463 / 150 distinct questions, with zero questions on both sides. Plotted as `fig_leakage_before_after.png`.

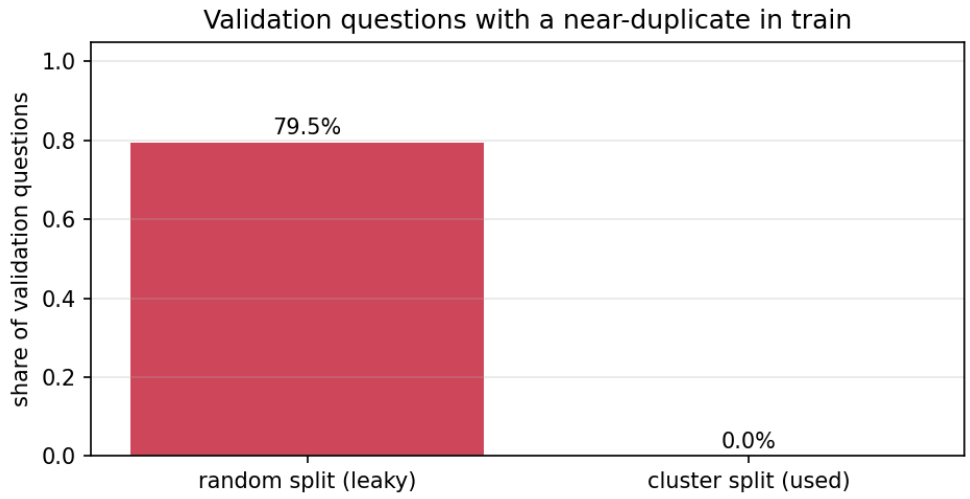


Figure: `fig_leakage_before_after.png`

An earlier attempt used TF-IDF similarity and only reduced residual leakage to 55.8%. Clustering with the same representation the retrieval components use closed the gap to zero.

Cluster splitting cannot guarantee label balance, so the distribution is printed and checked each run (train: B 391, C 372, D 295, A 290, E 252).

3.4 Data Augmentation

None. Given the duplicate density already present, synthetic paraphrases would have amplified memorisation rather than improved generalisation.

---

4. Tokenisation Strategy

4.1 Primary Tokeniser

WordPiece, paired with `bert-base-uncased` (vocabulary 30,522). It splits unknown words into known sub-word units, so domain terms ("Heidegger", "Newtonian") become sequences of familiar pieces rather than a single `[UNK]`. It also guarantees alignment with the pretrained embedding matrix - any other tokeniser would misalign every token id with the weights the model was pretrained on.

Each option is encoded as a **sentence pair** (`prompt, option`) so `token_type_ids` mark the boundary, the format BERT's next-sentence-prediction pretraining prepares it for.

4.2 Implementation Details

Measured token lengths across the 10,000 pairs: p50 56, p90 91, p99 131, max 200. `max_length` was set to 256, which truncates nothing. Padding every batch to that fixed length would be wasteful when the median pair is 56 tokens, so a custom collate function pads each batch to its own longest sequence. This cut mean padded length from 192 to 107 tokens, about 1.8x less computation per step, which made ten-epoch training affordable within Kaggle's GPU budget. Batches are shaped (`batch, 5, seq_len`), the layout `AutoModelForMultipleChoice` expects.

4.3 Experimental Comparison

tokeniser	model	validation MAP@3
WordPiece (30,522)	`bert-base-uncased`	**0.8729**
whitespace + regex (2,777 types)	from-scratch BiLSTM	0.8633
TF-IDF 1-2 grams (9,451 features)	Logistic Regression	0.7733

WordPiece was retained: highest scoring, and its uncased vocabulary suits a dataset where capitalization carries no label information.

---

5. Modelling and Experimentation

All models are evaluated on the same leakage-free split with the same MAP@3 implementation, unit-tested against the worked example in the competition description.

5.1 Model 1: Fine-Tuned Transformer

`bert-base-uncased` with a multiple-choice head: 12 layers, 768 hidden units, 12 heads, about 110M parameters.

The head is **not** a five-way classifier over the letters. Each option is paired with the prompt and scored independently by the same shared encoder, then a softmax over the five scalars gives a distribution. Two properties follow: the prediction is invariant to option order, because the letter is only an index applied after scoring; and each question yields **five** training signals against one shared scoring function, a strong inductive bias when only 2,000 questions are available.

setting	value	rationale
fine-tuning	full (~110M parameters)	see below

setting	value	rationale
optimizer	AdamW, lr 2e-5	decoupled weight decay, standard for BERT
schedule	linear warmup 10%, linear decay	avoids disturbing pretrained weights early
effective batch	16 (8 x 2 accumulation)	fits 16 GB at the target batch size
max length	256, dynamic padding	zero truncation, 1.8x less compute
precision	fp16 mixed	about 2x throughput on a T4
epochs	10 trained, epoch 6 selected	chosen by validation, not fixed in advance

No layers were frozen. An initial **LoRA** attempt (rank 8, 1.34M trainable parameters) was still climbing steeply at the final epoch with training loss stuck at 0.37 - under-trained rather than converged. LoRA constrains updates to a low-rank subspace and needs many more epochs. With 1,600 questions, updating all weights was affordable and scored higher.

Validation rose to 0.8729 at epoch 6 then sat within 0.0017 for four more epochs while training loss kept falling to 0.0241 - the signature of a model that has stopped generalising and begun memorising. That 0.0017 spread is far inside the noise of Section 6.5, so epoch 6 is a selection rule, not a claim of superiority over epoch 9. Top-1 accuracy is 0.7925.

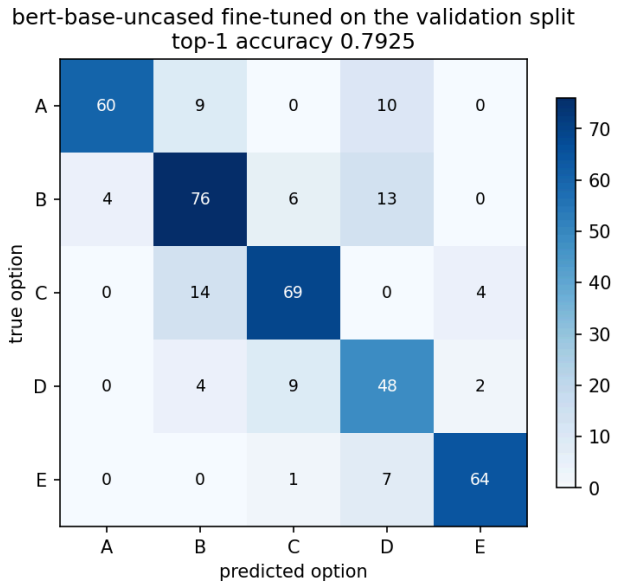


Figure: `fig_confusion_matrix.png`

Quantisation. Not used; inference was never the binding constraint.

5.2 Model 2: Custom Deep Learning Model

`ScratchMCQModel` is a siamese scorer built entirely from scratch: its own vocabulary, randomly initialised embeddings, no pretrained weights anywhere. Prompt and each option pass through a **shared** encoder to vectors `p` and `o`; the scorer receives `[p; o; |p-o|; p*o]` (1,024 features) and maps it through `Linear(1024, 128) + ReLU + Dropout(0.2) + Linear(128, 1)` to one score per option, softmaxed over the five.

Three deliberate choices. The **shared encoder** forces one general representation rather than two, and makes the model option-order invariant like the transformer. The `|p-o|` and `p*o` terms hand the scorer explicit dissimilarity and agreement signals instead of making the MLP learn comparison from scratch, following InferSent and Sentence-BERT. **Masked mean-pooling** ensures padding contributes nothing and short options are not penalised.

Trained with Adam at lr 1e-3 for 8 epochs, `MIN_FREQ = 2`, `MAX_LEN = 40`. The much higher learning rate than the transformer's 2e-5 is intentional: random weights must move far, pretrained weights must be nudged gently.

Encoder ablation. The scoring head is independent of how sequences are encoded, making it a controlled harness. Two further encoders were dropped in, holding vocabulary, head, optimiser, learning rate and epochs fixed:

encoder	mechanism	parameters	MAP@3
BiLSTM	recurrence with gating	-	**0.8633**
text-CNN	convolutions width 3/4/5, max-over-time	551,857	0.8462
mini transformer	multi-head self-attention, 2 blocks	691,329	0.8471

The mini transformer is the informative one: the **same mechanism as BERT**, every weight random. That separates two effects usually confounded:

- *architecture*: 0.8471 against the BiLSTM's 0.8633, both untrained. Self-attention did not help. With 1,600 questions and 40-token sequences there is neither the data nor the length for attention to pay off. The text-CNN at 0.8462 sits 0.0009 from the transformer, so those two are indistinguishable.
- *pretraining*: fine-tuned 0.8729 against untrained 0.8471, both transformers. The 0.026 difference is what pretraining bought with architecture held fixed.

That second figure is easy to overstate. The headline 0.4846 zero-shot versus 0.8729 fine-tuned makes pretraining look transformative, but against a randomly-initialised

transformer trained on the same data the gain is under three points. Most of the jump is task adaptation, not pretrained knowledge.

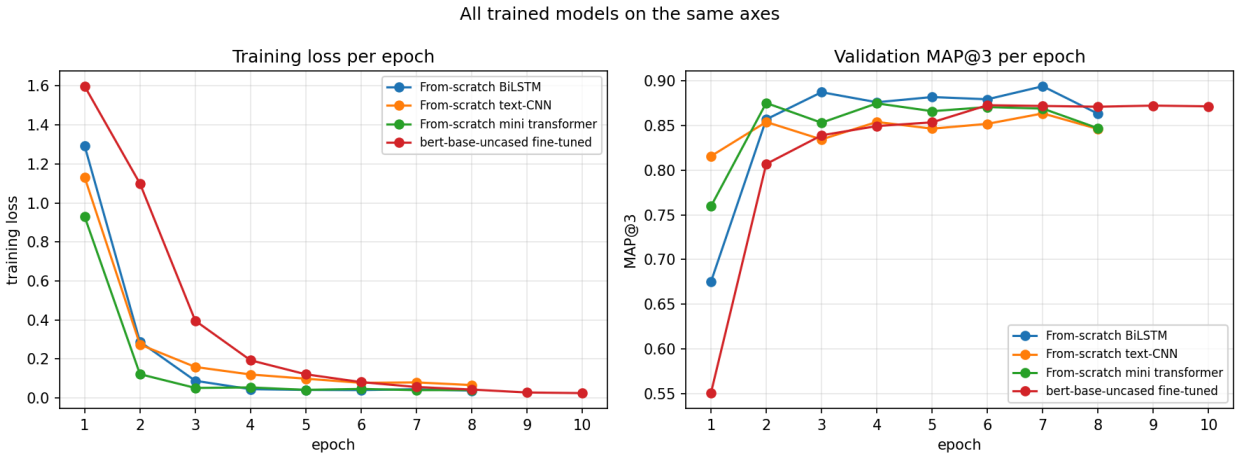


Figure: `fig_learning_curves_all_models.png`.

5.3 Model 3: RNN-Based Model

The sequence encoder inside Model 2 is a single-layer `nn.LSTM(100, 128, bidirectional=True)`, giving 256 dimensions per token, mean-pooled over valid positions.

A sequence model is appropriate because correctness depends on word order and composition, not a bag of words. “Humans exist within time” and “humans do not exist inside time, they are time” share almost all vocabulary and differ almost entirely in structure - the distinction TF-IDF cannot make, and why the BiLSTM (0.8633) beats TF-IDF with Logistic Regression (0.7733) by nine points.

Bidirectionality matters because meaning often depends on what follows (negations, qualifiers such as “does not have a defined beginning”). An LSTM was preferred over a vanilla RNN for its gating. The ablation supports the choice empirically: under identical conditions the recurrent encoder beats both alternatives. Its curve is noisy though - validation peaks at 0.8942 on epoch 7 and falls to 0.8633 on epoch 8 - so the ordering of the three encoders is indicative rather than settled.

5.4 Additional Experiments

system	description
TF-IDF + LogReg / SVM	1-2 gram features over `(prompt [SEP] option)`, classified correct/incorrect, then ranked
MiniLM zero-shot	frozen sentence embeddings; options ranked by cosine similarity to the prompt
RAG + Qwen2.5-0.5B-Instruct	retrieves the 3 most similar solved questions as few-shot context,

system	description
	scores each option by average per-token log-likelihood
Weighted Borda ensemble	rank fusion over TF-IDF, BiLSTM, BERT and RAG, each weighted by its own validation MAP@3
Hybrid retrieval layer	copies the correct-option text of the nearest training question; the model supplies ranks 2-3 and anything unmatched

Borda fusion was chosen because members produce scores on incomparable scales (probabilities, SVM margins, log-likelihoods), combining ranks avoids calibrating them.

---

6. Performance and Comparative Analysis

6.1 Evaluation Metrics

MAP@3. With one correct answer per question it reduces to the mean of `1 / rank` when the correct letter is in the top three, else 0. Implemented by hand and asserted against the competition's worked example before use. It is reported throughout rather than accuracy because it credits ranking the truth second or third - information accuracy discards.

6.2 Training Performance

Every figure below is generated by the notebook and written to `figures/`, so the report embeds the images the run produced rather than screenshots. Interactive versions are in Weights and Biases at wandb.ai/ambatwar-iitm/dlgenai-t226.

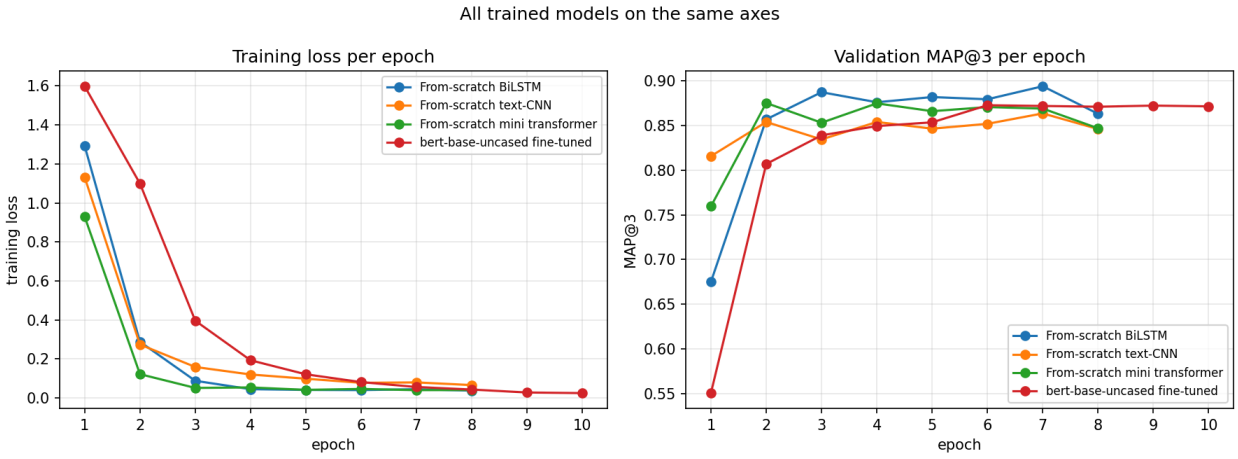


Figure: `fig_learning_curves_all_models.png`

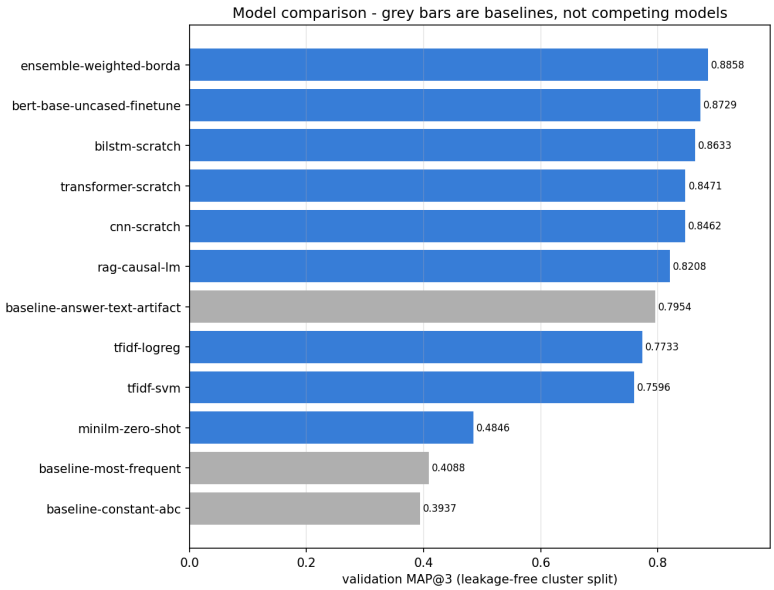


Figure: fig_model_comparison.png

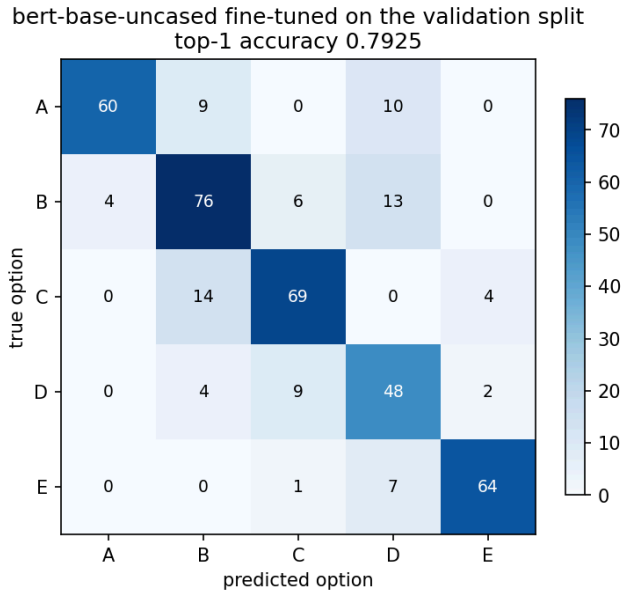


Figure: fig_confusion_matrix.png

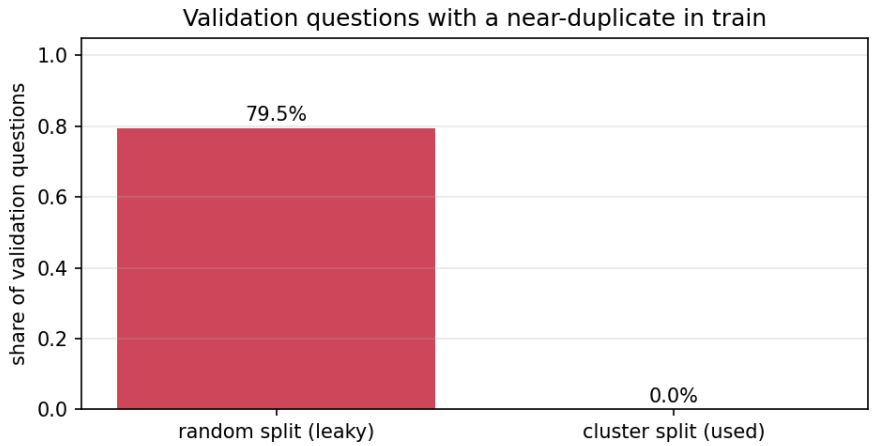
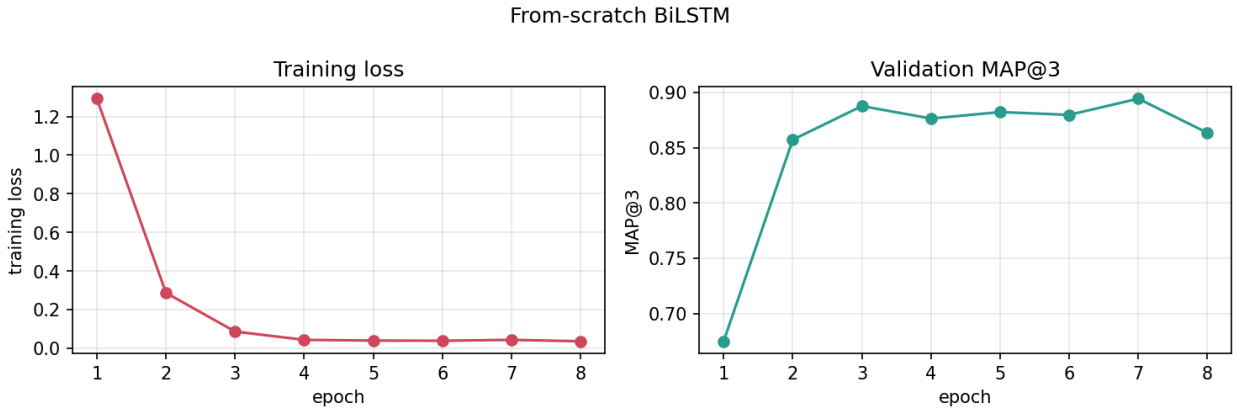
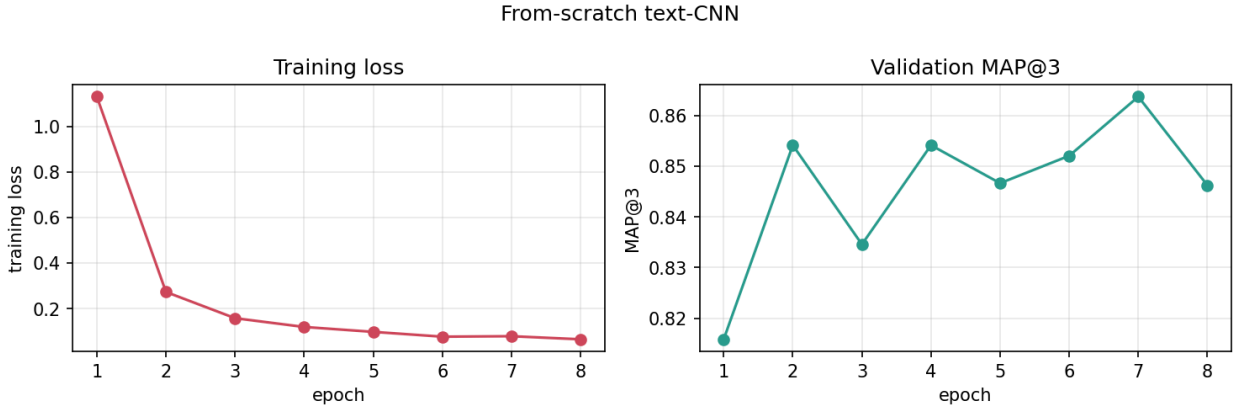
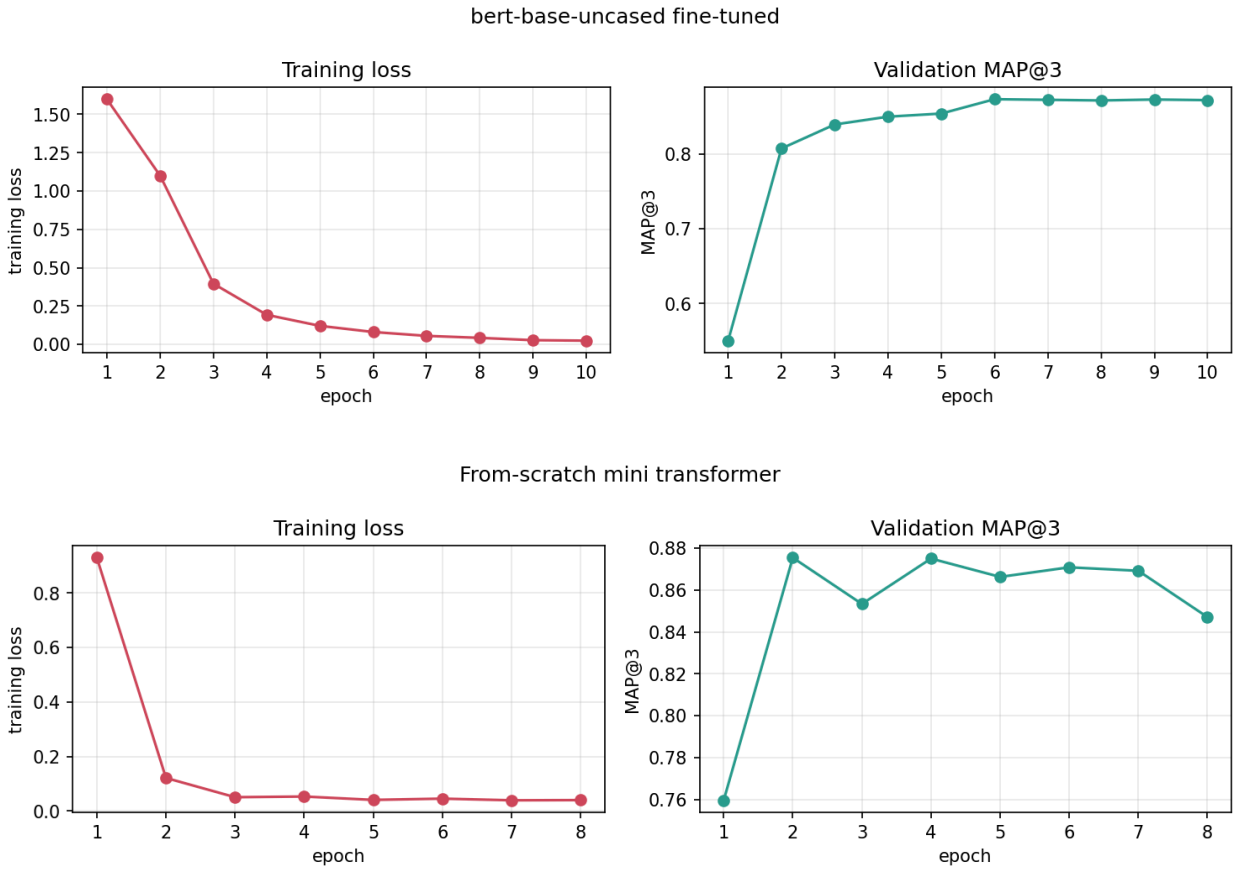


Figure: fig_leakage_before_after.png

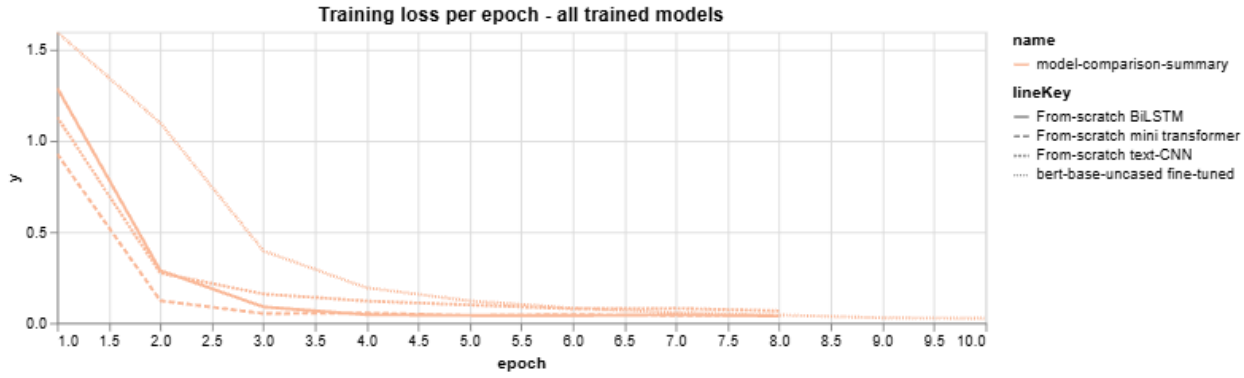
Down four per-model curve figures -

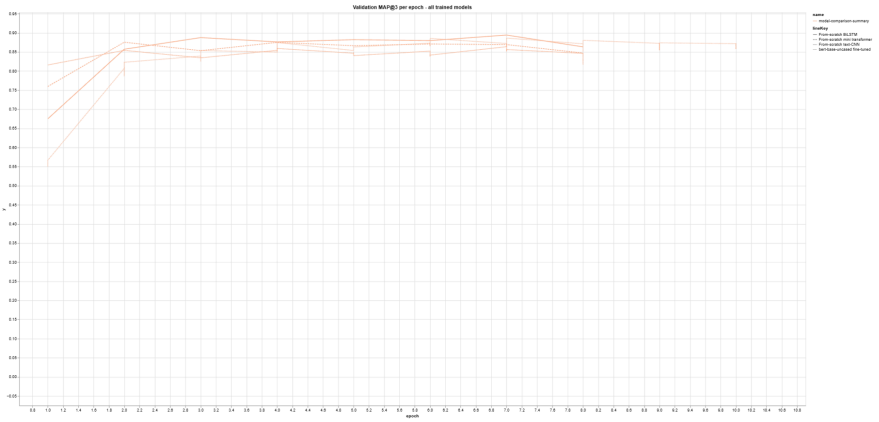






Additional experiment tracking was performed using Weights and Biases. The following figure shows the combined training loss curves logged during the experiments.





The transformer's training loss falls steadily to 0.0241 while validation stops improving after epoch 6 - the reason the best epoch is selected on validation rather than taking the last.

Tracking used W&B throughout: one run records the pipeline's step-by-step curves and submission artifacts; twelve short runs (group `model-comparison`) each log a single `val/map_at_3`; and a `model-comparison-summary` run carries the comparison table, bar chart, shared curves, confusion matrix and leakage chart.

6.3 Comparative Report

#	model	family	MAP@3
1	**Ensemble (weighted Borda)**	fusion	**0.8858**
2	`bert-base-uncased` fine-tuned	pretrained transformer	0.8729
3	From-scratch BiLSTM	custom / RNN	0.8633
4	From-scratch mini transformer	custom / self-attention	0.8471
5	From-scratch text-CNN	custom / convolutional	0.8462
6	RAG + Qwen2.5-0.5B	retrieval + causal LM	0.8208
7	**Answer-text artifact**	**baseline, reads no question**	**0.7954**

#	model	family	MAP@3
8	TF-IDF + Logistic Regression	classical ML	0.7733
9	TF-IDF + linear SVM	classical ML	0.7596
10	MiniLM (zero-shot)	pretrained, frozen	0.4846
11	Constant `B C D`	baseline	0.4088
12	Constant `A B C`	baseline	0.3937

Row 7 is not a model. It ranks options solely by whether an option's text was ever a correct answer in training, never reading the prompt. It has no parameters and no training. It is in the table because it belongs there: it outranks both TF-IDF variants and the zero-shot encoder, all of which do process text.

The honest baseline is therefore 0.7954, not 0.3937. The portion of any score attributable to reading the question is the distance above that line:

model	MAP@3	above the artifact baseline
Ensemble	0.8858	+0.090
fine-tuned transformer	0.8729	+0.078
from-scratch BiLSTM	0.8633	+0.068
mini transformer	0.8471	+0.052
text-CNN	0.8462	+0.051
RAG	0.8208	+0.025
TF-IDF + LogReg	0.7733	**-0.022**

TF-IDF lands *below* the line: a lookup table beats it, so its 0.7733 is not evidence of language understanding. Measuring against the 0.3937 constant baseline, as an earlier draft did, credits every model with work the dataset performed for it.

Trade-offs.

Pretraining is decisive only when adapted. MiniLM as-is scores 0.4846, barely above 0.4088; the same class fine-tuned reaches 0.8729. Section 5.2 qualifies this: against a randomly-initialised transformer the gain is only 0.026.

Cost versus accuracy is nearly flat here. The BiLSTM reaches 98.9% of the transformer's score in 8 seconds against 7.5 minutes - roughly 55x cheaper for 0.010 MAP@3. Measured above the artifact baseline it captures 0.068 of the transformer's 0.078, or 87% of the genuine learning at 2% of the cost. Where compute is constrained it is the better

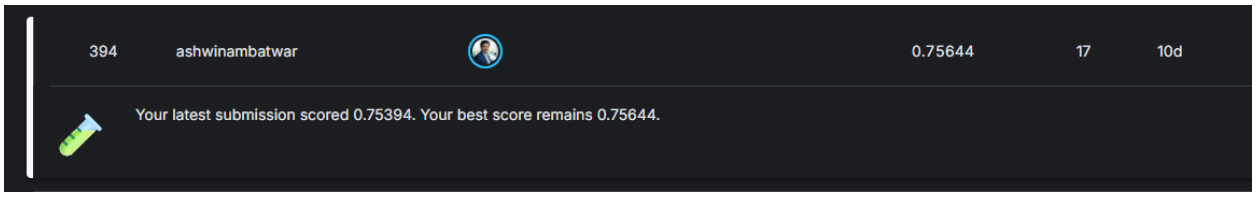
engineering choice. The claim must stay careful though: 0.010 is inside the noise of Section 6.5, so the two are **not cleanly separable** under this protocol, which is not the same as equivalent.

Structure beats surface matching. All three from-scratch encoders beat both TF-IDF variants, so this is a property of sequence modelling generally, not of the LSTM.

The ensemble won, narrowly. 0.8858 edges 0.8729 by 0.013. An earlier run of the same code placed it below the best single model, so this ordering is not stable.

6.4 Kaggle Performance and the Validation Gap

Best leaderboard score: 0.75664 MAP@3 (Kaggle rank: **394**), against the 0.73 cutoff.



The final notebook version, whose numbers this report quotes, scored **0.75394**. Both are given deliberately, the gap is a measurement, analysed below.

submitted system	validation MAP@3	Kaggle MAP@3
Ensemble + retrieval, run A	0.8742	**0.75664**
Ensemble + retrieval, run B (final)	**0.8858**	0.75394
RAG + Qwen2.5-0.5B	0.8208	0.75436
`bert-base-uncased`, 5 epochs	0.8371	0.75228
`bert-base-uncased`, 8 epochs	0.8829	0.74937
`roberta-base`, 5 epochs	0.8633	0.74438
`roberta-base`, 3 epochs	0.8425	0.73940

Validation spans 0.82 to 0.89 while the leaderboard spans 0.739 to 0.757. A 0.07 local improvement bought under 0.02, and the ordering inverts repeatedly: RAG, with the worst validation score of the group, beat two models that validated far better.

The first two rows are the sharpest evidence, being **the same code, seed and data run twice** with only fp16 non-determinism differing. Run B validated 0.0116 *higher* and scored 0.0027 *lower*.

Cause: the test set is half memorisation. Comparing all 500 test questions against `train.csv` by exact string match:

relationship to `train.csv`	share of test	recoverable by lookup?
identical question (same prompt *and* same 5 options)	**52.8%** (264/500)	yes
same prompt, different options	0.8% (4/500)	partly
different prompt, same option set	38.8% (194/500)	no - a different question
no match at all	7.6% (38/500)	no

The 38.8% that merely reuse an option set are genuinely different questions: the dataset recycles distractor pools across prompts, which is why an embedding-similarity measure initially suggested a misleading 93.4% "near-duplicate" rate. Exact matching corrected that to 52.8%.

The validation split deliberately removes near-duplicates, so it measures reasoning. The test set is roughly 53% recall and 47% reasoning, so the leaderboard blends the two and rewards memorisation. That is why RAG, which explicitly retrieves from training data, outperformed models with far better validation scores.

The final submission makes this concrete. A hybrid layer looks up the nearest training question and copies its correct-option text, leaving the model to supply ranks 2-3 and answer whatever retrieval cannot match. On both final runs it set rank 1 for exactly the same **466 of 500 questions (93.2%)** - retrieval is deterministic, so those are identical between runs by construction. Only 34 questions plus the rank-2 and rank-3 slots can move, which bounds achievable variation at a few thousandths and matches the observed 0.0027. It is also why nine submissions spanning five architectures all landed within 0.018 of each other.

An honest estimate of reasoning ability. If the memorisable half is answered essentially perfectly, the implied MAP@3 on the remaining 47% is `(0.75664 - 0.528) / 0.472 ~= 0.48`, against a 0.3937 random baseline: only modestly above chance on genuinely novel questions. This is an inference from the leaderboard score and the composition above, not a measured value, since test labels are not public. It is consistent with the deployed model's observed behaviour on questions outside this dataset.

6.5 Error Analysis

Leakage in the original protocol. The first design was a stratified random split, under which 79.5% of validation questions had a twin in training. RAG scored a perfect 1.0000 and the ensemble 1.0000 - obviously invalid. Cluster splitting reduced this to 0.0% and dropped every score to a believable range, Although the score became lower after fixing the

split, the experiment became more reliable. The clearest casualty was TF-IDF + SVM, which fell from 0.9721 to 0.7596 once it could no longer recall questions it had already seen.

The labelling artifact, quantified. Rather than assert it, the notebook derives it:

stage	result
texts used as correct answer / distractor / both	313 / 1,271 / **0**
validation questions with exactly one flagged option	255 of 400
flagged option is the true answer	always, by construction
rank 1 / 2 / 3 / missed	293 / 31 / 29 / 47
credit	(293 x 1) + (31 x 0.5) + (29 x 1/3) = 318.17
MAP@3	318.17 / 400 = **0.7954**

The 293 rank-1 hits are 255 identified with certainty by the flag plus 38 guessed correctly by the letter-frequency fallback on the 145 unflagged questions.

Crucially the cluster split does **not** remove this. It removes duplicate *prompts*; 64% of validation questions still carry an answer text seen during training. That is why validation reads 0.87 at a 0.0% duplicate rate.

Over-training. Pushing the transformer from 5 to 8 epochs raised validation from 0.8371 to 0.8829 but *lowered* the leaderboard from 0.75228 to 0.74937. Fitting the training distribution harder improved the memorisation-flavoured metric and hurt the real one.

Run-to-run variance, measured on both sides. Running the finished notebook twice:

quantity	run A	run B	difference
fine-tuned transformer	0.8650	0.8729	+0.008
ensemble (submitted)	0.8742	0.8858	+0.012
text-CNN	0.8358	0.8462	+0.010
Kaggle leaderboard	**0.75664**	**0.75394**	**-0.003**

Validation noise is roughly +/- 0.01, leaderboard noise +/- 0.003, and here they move in opposite directions. This governs how several results may be read. Claims inside the band are stated as “not separable” rather than as orderings: BiLSTM against the transformer (0.010), ensemble against the transformer (0.013), text-CNN against the mini transformer (0.0009), transformer epochs 6 to 10 (0.0017). Every submission between 0.7539 and 0.7566 should likewise be read as one result.

Differences that **do** survive: every model over the artifact baseline (0.025 to 0.090), sequence models over TF-IDF (nine points), fine-tuning over zero-shot (0.39), and every model over the constant baselines.

---

7. Conclusion and Future Work

7.1 Key Learnings

1. **The main lesson from this project was that the evaluation method itself needed to be checked carefully.** More value came from diagnosing the validation split than from any architectural change. A metric that cannot be trusted makes every subsequent decision arbitrary.
2. **Measure how much of your own score is an artifact.** The question-blind baseline at 0.7954 was uncomfortable to compute and is the measurement that makes every other number in this report trustworthy.
3. **Pretraining plus adaptation, not pretraining alone.** 0.4846 zero-shot versus 0.8729 fine-tuned, refined by the mini transformer at 0.8471: most of that gap is task adaptation, not pretrained knowledge.
4. **Inductive bias substitutes for data.** The per-option scoring head extracts five training signals per question against shared weights - decisive with 2,000 examples.
5. **Quantify noise before interpreting differences.** Several headline comparisons here fall inside +/- 0.01. Knowing which survive, and saying so, matters more than the leaderboard position.

7.2 Challenges Faced

Data leakage. Detected by an in-notebook diagnostic, solved by cluster-based splitting. The first attempt used TF-IDF similarity and reached only 55.8% residual; switching to the embeddings used downstream achieved 0.0%.

GPU memory. Three large models (BERT, MiniLM, Qwen2.5-0.5B) plus two BiLSTMs in one notebook. Each is explicitly freed with `del` and `torch.cuda.empty_cache()` once its outputs are cached.

Compute budget. Dynamic per-batch padding cut compute per step by about 1.8x, making ten-epoch training viable within Kaggle's daily quota.

Misleading similarity metrics. An embedding estimate suggested 93.4% of the test set was near-duplicated; exact matching showed 52.8%. The first figure had been used to project a leaderboard score near 0.98, which actual submissions contradicted by more than 0.2.

7.3 Areas for Improvement

1. **Cross-validation instead of a single split.** 400 validation rows give roughly +/- 0.01 noise, wider than most differences this report reports. Five-fold CV over clusters would make them meaningful.

2. **Train on external MCQ corpora.** RACE, ARC or OpenBookQA would target genuine multiple-choice reasoning rather than this dataset's artifact, and would likely improve out-of-domain behaviour - though, given the test composition, probably not the leaderboard score.

7.4 Deployment

The fine-tuned model is published at `huggingface.co/ashwinambatwariitm/dlgenai-t22026-mcq` with documentation describing architecture, training recipe, both evaluation numbers and the limitations above. A Streamlit application provides an interactive demo: the user enters a question and five options and receives the predicted letter, the MAP@3-format top-3, and a confidence bar per option. It offers two example loaders - one drawing verbatim training questions, one drawing the 38 test questions that appear nowhere in training - so a reviewer can observe the confidence collapse between memorised and genuinely unseen questions directly.

---

8. References

1. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL-HLT. arXiv:1810.04805
2. Liu, Y. et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach.* arXiv:1907.11692
3. He, P., Gao, J., Chen, W. (2023). *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing.* ICLR. arXiv:2111.09543
4. Reimers, N., Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP. arXiv:1908.10084
5. Conneau, A. et al. (2017). *Supervised Learning of Universal Sentence Representations from Natural Language Inference Data.* EMNLP. arXiv:1705.02364
6. Hu, E. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685
7. Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization.* ICLR.